

Day 33 – Docker Compose: Multi-Container Basics

Task 1: Install & Verify Docker Compose

Install

`sudo curl -L`

`"https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose`

Apply executable permissions

`sudo chmod +x /usr/local/bin/docker-compose`

```
ubuntu@ip-172-31-81-141:~$ sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0      0     0
100 31.36M 100 31.36M    0     0 111.2M      0    0     0
ubuntu@ip-172-31-81-141:~$ sudo chmod +x /usr/local/bin/docker-compose
ubuntu@ip-172-31-81-141:~$
```

Check Docker Compose Version

None

`docker-compose --version`

```
ubuntu@ip-172-31-81-141:~$ docker-compose --version
Docker Compose version v5.1.4
ubuntu@ip-172-31-81-141:~$
```

Verify Docker is Running

None

`docker ps`

If containers list appears, Docker is working.

Task 2: Your First Compose File

Step 1: Create Project Directory

```
None
mkdir compose-basics
cd compose-basics
```

Step 2: Create docker-compose.yml

```
None
version: '3.9'

services:
  nginx:
    image: nginx:latest
    container_name: nginx-compose
    ports:
      - "8080:80"
```

Step 3: Start Container

```
None
docker compose up -d
```

Output:

```
None
Creating nginx-compose ... done
```

Step 4: Verify Running Container

None

`docker compose ps`

```
ubuntu@ip-172-31-41-65:~/compose-basics$ docker compose up -d
WARN[0000] /home/ubuntu/compose-basics/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 10/10
 ✓ nginx Pulled
 ✓ c82e83b01f69 Pull complete
 ✓ ac5e3151b8c0 Pull complete
 ✓ 99181f19640f Pull complete
 ✓ 8da80f8205ea Pull complete
 ✓ 3461bb328618 Pull complete
 ✓ 868d78dceaad Pull complete
 ✓ 72c03230f136 Pull complete
 ✓ 4b84eff91c61 Download complete
 ✓ bcac76e1b5c9 Download complete
[+] Running 2/2
 ✓ Network compose-basics_default Created
 ✓ Container nginx-compose Started
ubuntu@ip-172-31-41-65:~/compose-basics$ docker compose ps
WARN[0000] /home/ubuntu/compose-basics/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                  SERVICE    CREATED          STATUS          PORTS
nginx-compose       nginx:latest         "/docker-entrypoint..." nginx      13 seconds ago   Up 12 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
ubuntu@ip-172-31-41-65:~/compose-basics$
```

or

None

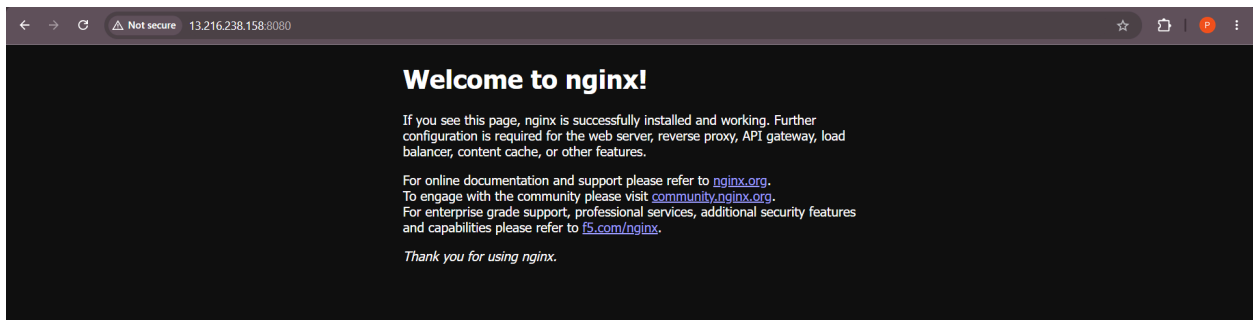
`docker ps`

Step 5: Access Nginx

Browser:

None

`http://<SERVER-IP>:8080`



Example:

None

`http://54.123.45.67:8080`

You should see:

None

```
Welcome to nginx!
```

Step 6: Stop and Remove

None

```
docker compose down
```

Verify:

None

```
docker ps
```

No containers should be running.

Task 3: WordPress + MySQL Setup

Step 1: Create Directory

None

```
mkdir wordpress-compose  
cd wordpress-compose
```

Step 2: Create docker-compose.yml

None

```
version: '3.9'  
  
services:
```

```
db:
  image: mysql:8.0
  container_name: mysql-db

  restart: always

  environment:
    MYSQL_ROOT_PASSWORD: root123
    MYSQL_DATABASE: wordpress
    MYSQL_USER: wpuser
    MYSQL_PASSWORD: wpuser123

  volumes:
    - mysql_data:/var/lib/mysql

wordpress:
  image: wordpress:latest
  container_name: wordpress-app

  restart: always

  ports:
    - "8080:80"

  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wpuser123
    WORDPRESS_DB_NAME: wordpress

  depends_on:
    - db

volumes:
  mysql_data:
```

Step 3: Start Services

None

```
docker compose up -d
```

```
ubuntu@ip-172-31-41-65:~/wordpress-compose$ docker compose up -d
WARN[0000] /home/ubuntu/wordpress-compose/docker-compose.yml: the
[+] Running 40/40
  ✓ db Pulled
  ✓ wordpress Pulled

[+] Running 4/4
  ✓ Network wordpress-compose_default      Created
  ✓ Volume wordpress-compose_mysql_data   Created
  ✓ Container mysql-db                     Started
  ✓ Container wordpress-app                Started
ubuntu@ip-172-31-41-65:~/wordpress-compose$
```

Check:

None

```
docker ps
```

Expected:

None

wordpress-app

mysql-db

```
ubuntu@ip-172-31-41-65:~/wordpress-compose$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
e2962610c9a9   wordpress:latest "docker-entrypoint.s..." 58 seconds ago Up 58 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  wordpress-app
c6e404ef07e9   mysql:8.0      "docker-entrypoint.s..." About a minute ago Up 58 seconds  3306/tcp, 33060/tcp                mysql-db
ubuntu@ip-172-31-41-65:~/wordpress-compose$
```

Step 4: Open WordPress

Browser:

None

<http://<SERVER-IP>:8080>

You'll see the WordPress installation page.

Choose:

- Language
- Site Name
- Username
- Password
- Email

Click:

None

[Install WordPress](#)

Step 5: Verify Network

Compose automatically creates a network.

Check:

None

```
docker network ls
```

```
ubuntu@ip-172-31-41-65:~/wordpress-compose$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d25d138ac78e        bridge              bridge              local
55d3b0ad70c9        host                host                local
e0796ad58164        none                null                local
3b063b7bea80        wordpress-compose_default bridge              local
ubuntu@ip-172-31-41-65:~/wordpress-compose$
```

Inspect:

None

```
docker network inspect wordpress-compose_default
```

```
{
  "Containers": {
    "c6e404ef07e92805cd354d1558445256eae06ad153b1ff003f45a54281aa6198": {
      "Name": "mysql-db",
      "EndpointID": "cdbfbfb919412f19fa2ea766c6c0dd9b4332f724bcc8d410eea7195e6342aae7",
      "MacAddress": "ea:b5:8d:b7:3a:00",
      "IPv4Address": "172.18.0.2/16",
      "IPv6Address": ""
    },
    "e2962610c9a9c7bc59af5f06e181f578ac79e5e793ba5532d59cf4c3cb867665": {
      "Name": "wordpress-app",
      "EndpointID": "a67145b01f8a304a667246bd1bbeca807e64020a4a0d76d13ea5f9a3fbb56c26",
      "MacAddress": "16:a3:11:eb:fa:4b",
      "IPv4Address": "172.18.0.3/16",
      "IPv6Address": ""
    }
  }
}
```

```
ubuntu@ip-172-31-41-65:~/wordpress-compose$ docker network inspect
wordpress-compose_default
```

```
[
```

```
{
```

```
  "Name": "wordpress-compose_default",
```

```
  "Id": "3b063b7bea802936a9172be5fdf4c20fc21d5829f1ad25c4af65c049f539b0a3",
```

```
  "Created": "2026-06-20T13:43:02.781201496Z",
```

```
  "Scope": "local",
```

```
"Driver": "bridge",

"EnableIPv4": true,

"EnableIPv6": false,

"IPAM": {

  "Driver": "default",

  "Options": null,

  "Config": [

    {

      "Subnet": "172.18.0.0/16",

      "IPRange": "",

      "Gateway": "172.18.0.1"

    }

  ]

},

"Internal": false,

"Attachable": false,

"Ingress": false,

"ConfigFrom": {

  "Network": ""

},

"ConfigOnly": false,

"Options": {},

"Labels": {

  "com.docker.compose.config-hash":

    "2ee3ef1beb61075fdd5d6fc1a333d52160a0254dad2738af897eee58106444c7",
```

```
"com.docker.compose.network": "default",

"com.docker.compose.project": "wordpress-compose",

"com.docker.compose.version": "2.40.3"

},

"Containers": {

  "c6e404ef07e92805cd354d1558445256eae06ad153b1ff003f45a54281aa6198": {

    "Name": "mysql-db",

    "EndpointID":

"cdbfbfb919412f19fa2ea766c6c0dd9b4332f724bcc8d410eea7195e6342aae7",

    "MacAddress": "ea:b5:8d:b7:3a:00",

    "IPv4Address": "172.18.0.2/16",

    "IPv6Address": ""

  },

  "e2962610c9a9c7bc59af5f06e181f578ac79e5e793ba5532d59cf4c3cb867665": {

    "Name": "wordpress-app",

    "EndpointID":

"a67145b01f8a304a667246bd1bbeca807e64020a4a0d76d13ea5f9a3fbb56c26",

    "MacAddress": "16:a3:11:eb:fa:4b",

    "IPv4Address": "172.18.0.3/16",

    "IPv6Address": ""

  }

},

"Status": {

  "IPAM": {

    "Subnets": {
```

```
"172.18.0.0/16": {  
  "IPsInUse": 5,  
  "DynamicIPsAvailable": 65531  
}  
}  
}  
}  
}  
}  
]  
ubuntu@ip-172-31-41-65:~/wordpress-compose$
```

Notice:

None
db
wordpress

Service names become DNS names.

WordPress connects to MySQL using:

None
db

not IP addresses.

Step 6: Verify Volume Persistence

Check volume:

None

```
docker volume ls
```

```
ubuntu@ip-172-31-41-65:~/wordpress-compose$ docker volume ls
DRIVER      VOLUME NAME
local       adcd4a10ac794d32d1cd724fd9009b265b955fc40249e72555bd1defc87f08d1
local       wordpress-compose_mysql_data
ubuntu@ip-172-31-41-65:~/wordpress-compose$
```

Expected:

None

```
wordpress-compose_mysql_data
```

Stop Containers

None

```
docker compose down
```

Volume still exists.

Start Again

None

```
docker compose up -d
```

```
ubuntu@ip-172-31-41-65:~/wordpress-compose$ docker compose up -d
WARN[0000] /home/ubuntu/wordpress-compose/docker-compose.yml: the attribute `version` is
[+] Running 3/3
 ✓ Network wordpress-compose_default Created
 ✓ Container mysql-db Started
 ✓ Container wordpress-app Started
```

If you're running WordPress with Docker and have already completed the installation, the login URL is:

None

```
http://<SERVER-IP>:8080/wp-admin
```

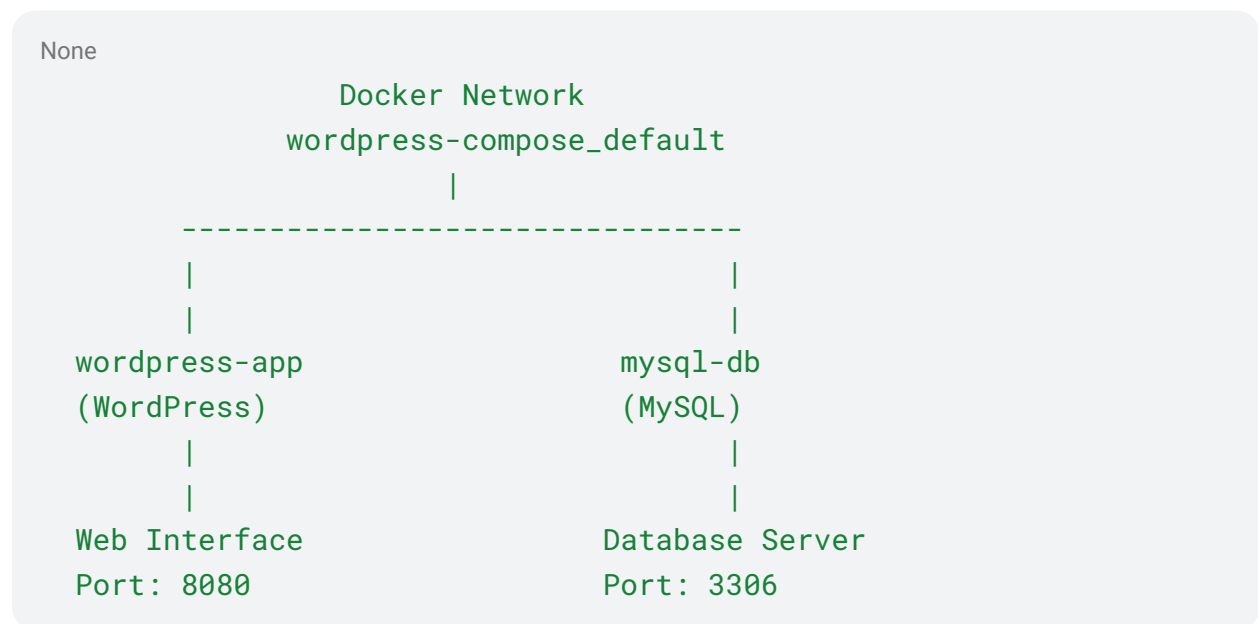
Visit WordPress again.

Your site data should still be present because MySQL data is stored in the named volume.

Done.....

Explanation for wordpress+mysql

Your Docker Compose file creates **two separate containers**:



WordPress and MySQL Database Creation Flow

When Docker Compose starts, it creates two containers:

- **wordpress-app** → Runs the WordPress application.
- **mysql-db** → Runs the MySQL database server.

Database Creation

During MySQL container startup, the environment variables create an empty database:

```
None
MYSQL_DATABASE: wordpress
MYSQL_USER: wpuser
MYSQL_PASSWORD: wpuser123
```

This creates:

```
None
MySQL Server
  |
  └─ wordpress database
      |
      └─ (empty database)
```

At this stage, WordPress tables such as `wp_users`, `wp_posts`, and `wp_options` do not exist.

WordPress Installation Process

When the WordPress container starts, it connects to MySQL using:

```
None
Database Host: db:3306
Database Name: wordpress
Username: wpuser
Password: wpuser123
```

Docker internally resolves the service name `db` to the MySQL container IP address.

When the user opens:

```
None
http://SERVER-IP:8080
```

and completes the WordPress installation, WordPress executes SQL commands to create the required database tables.

Example tables created:

None

wordpress database

```
|— wp_users
|— wp_usermeta
|— wp_posts
|— wp_postmeta
|— wp_comments
|— wp_options
```

The table creation structure is already included inside the WordPress application code. During installation, WordPress reads this schema and sends SQL `CREATE TABLE` commands to MySQL.

User Creation

When an administrator account is created during installation:

- User information is stored in the `wp_users` table.
- User roles and permissions are stored in the `wp_usermeta` table.
- Passwords are stored as encrypted hashes, not plain text.

Example:

None

`wp_users`

ID	username	password
1	admin	hashed_password

Data Persistence

MySQL stores all WordPress data inside:

None

`/var/lib/mysql`

Docker maps this location to the named volume:

None

`mysql_data`

Because of this volume, database data remains available even if containers are removed and recreated.

Complete Flow

None

```
docker compose up -d
  |
  v
MySQL container starts
  |
  v
Creates empty wordpress database
  |
  v
WordPress container connects to MySQL
  |
  v
User completes WordPress installation
  |
  v
WordPress creates database tables
  |
  v
Users, posts, settings are stored in MySQL
  |
  v
Data is saved permanently using mysql_data volume
```

Summary:

MySQL creates the empty database when the container starts. WordPress creates the required tables during the first installation. All WordPress content, users, and settings are stored inside the MySQL database.

If you delete both containers (**wordpress-app** and **mysql-db**) but keep the volume (**mysql_data**), you can recreate them and recover the same database.-----

Scenario: Delete both containers and recreate them

Step 1: Check current containers

None

```
docker ps
```

Example:

None

CONTAINER ID	NAME	IMAGE
xxxxx	wordpress-app	wordpress:latest
xxxxx	mysql-db	mysql:8.0

Step 2: Stop and remove containers

Using Docker Compose:

None

```
docker compose down
```

This removes:

None

wordpress-app	✗ deleted
mysql-db	✗ deleted

But keeps:

None

mysql_data  exists

Step 3: Verify the volume still exists

Run:

None

```
docker volume ls
```

Output:

None

DRIVER	VOLUME NAME
--------	-------------

local	wordpress-compose_mysql_data
-------	------------------------------

Your database files are still inside this volume.

Step 4: Check the volume details (optional)

None

```
docker volume inspect wordpress-compose_mysql_data
```

Example:

None

```
[
```

```
{
```

```
        "Name": "wordpress-compose_mysql_data",  
        "Mountpoint":  
        "/var/lib/docker/volumes/wordpress-compose_mysql_data/_data"  
    }  
]
```

This location contains your MySQL data.

Step 5: Recreate containers

Run:

```
None  
docker compose up -d
```

Docker creates new containers:

```
None  
wordpress-app  ✓ new container  
mysql-db      ✓ new container
```

But MySQL attaches the existing volume:

```
None  
mysql-db container  
|  
|  
v
```

```
/var/lib/mysql
```

```
|
```

```
|
```

```
v
```

```
wordpress-compose_mysql_data
```

```
|
```

```
|
```

```
v
```

```
Old database
```

Step 6: Verify containers are running

```
None
```

```
docker ps
```

Expected:

```
None
```

```
CONTAINER NAME
```

```
wordpress-app
```

```
mysql-db
```

Step 7: Verify the database is restored

Enter MySQL:

```
None  
docker exec -it mysql-db bash
```

Login:

```
None  
mysql -u wpuser -p
```

Password:

```
None  
wpuser123
```

Select database:

```
None  
USE wordpress;
```

Check tables:

```
None  
SHOW TABLES;
```

You should see:

```
None  
wp_users  
  
wp_posts  
  
wp_options  
  
wp_comments
```

```
wp_usermeta
```

```
...
```

Step 8: Verify WordPress site

Open:

```
None
```

```
http://SERVER-IP:8080
```

or admin:

```
None
```

```
http://SERVER-IP:8080/wp-admin
```

Your previous:

- Admin user ☒
- Posts ☒
- Pages ☒
- Settings ☒
- Plugins stored in database ☒

will still exist.

Important Note

Your current `docker-compose.yml` only persists MySQL:

```
None
```

```
volumes:
```

```
- mysql_data:/var/lib/mysql
```

So:

Restored:

- ✓ Users
- ✓ Posts
- ✓ Pages
- ✓ Settings
- ✓ WordPress configuration stored in database

Not guaranteed:

- ✗ Themes files
- ✗ Plugin files
- ✗ Uploaded images/media files

Those are stored inside the WordPress container:

None

```
/var/www/html/wp-content
```

For a production setup, add another volume:

None

```
wordpress:
```

```
  volumes:
```

```
    - wordpress_data:/var/www/html
```

```
volumes:
```

```
  mysql_data:
```

```
  wordpress_data:
```

Then both database and WordPress files survive container deletion.

Task 4: Practice Docker Compose Commands

Start in Detached Mode

None

```
docker compose up -d
```

View Running Services

None

```
docker compose ps
```

View All Logs

None

```
docker compose logs
```

Live logs:

None

```
docker compose logs -f
```

Logs of Specific Service

MySQL:

None

```
docker compose logs db
```

WordPress:

None

```
docker compose logs wordpress
```

Stop Without Removing

None

```
docker compose stop
```

Restart:

None

```
docker compose start
```

Remove Containers and Network

None

```
docker compose down
```

Remove Everything Including Volumes

None

```
docker compose down -v
```

 This deletes database data permanently.

Rebuild Images

None

```
docker compose up -d --build
```

or

None

```
docker compose build
```

Task 5: Environment Variables

Create .env File

None

```
touch .env
```

Add:

None

```
MYSQL_ROOT_PASSWORD=root123
MYSQL_DATABASE=wordpress
MYSQL_USER=wpuser
MYSQL_PASSWORD=wpuser123
```

Update docker-compose.yml

None

```
version: '3.9'
```

```
services:
```

```
  db:
```

```
image: mysql:8.0

environment:
  MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  MYSQL_DATABASE: ${MYSQL_DATABASE}
  MYSQL_USER: ${MYSQL_USER}
  MYSQL_PASSWORD: ${MYSQL_PASSWORD}

volumes:
  - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

Verify Variables

Check final compose configuration:

```
None
docker compose config
```

You should see actual values substituted from `.env`.

Example:

```
None
MYSQL_ROOT_PASSWORD: root123
MYSQL_DATABASE: wordpress
```

Final Verification Checklist

☒ Docker Compose installed

- ✓ Nginx deployed with Compose
- ✓ WordPress + MySQL deployed
- ✓ Named volume created
- ✓ Data persists after restart
- ✓ Used Compose commands
- ✓ Environment variables loaded from `.env`

Project Structure

None

```
Day-33-Docker-Compose/  
|  
├── compose-basics/  
│   └── docker-compose.yml  
|  
├── wordpress-compose/  
│   ├── docker-compose.yml  
│   └── .env  
|  
└── day-33-compose.md
```

This completes Day 33 and gives you hands-on experience with Docker Compose, networking, volumes, service discovery, and environment variables—skills commonly asked in DevOps interviews.